

Exception Handling Part-1

DAY 77
25/09/2025

What is an Exception?

An Exception is an unexpected event or error that occurs during program execution, which disrupts the normal flow of instructions.

It usually happens due to runtime issues like

- Invalid input
- Network errors
- File not found
- Division By Zero
- Null pointer access

Example

```
public class Example {  
    public static void main (String[] args) {  
        int a = 10, b = 0;  
        System.out.println(a/b); // Division By 0 Exception  
    }  
}
```

output: Exception in thread "main" java.lang.ArithmeticException: /by zero

What is Exception Handling?

• Exception Handling is a mechanism to detect, handle, and recover from exceptions in a controlled way, without crashing the program.

• In Java, this is done using:

- try → code that might throw exception
- catch → handle the exception
- finally → code that always executes (cleanup)
- throw → explicitly throw an exception
- throws → declare exceptions in Method signature

java

```
public class Example {  
    public static void main(String[] args) {  
        try {  
            int a = 10, b = 0;  
            System.out.println(a / b); // risky code  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero!");  
        } finally {  
            System.out.println("End of program.");  
        }  
    }  
}
```

Output:

```
Cannot divide by zero!  
End of program.
```

Why do we need Exception Handling?

Without exception handling → program crashes → bad user experience
With exception handling → program recovers gracefully.

Reasons we need it

1. Graceful failure - Instead of crashing, show meaningful error messages
 - Example: "File not found, please check path."
2. Separation of error-handling logic - keeps the business logic clean
3. Resource management - Ensures resources (files, DB connections) are closed even if errors occur (finally or try-with-resources).
4. Debugging help - Exceptions give stack trace → easier to find bug
5. Robust applications - Real-world apps deal with network failures, file corruption, invalid inputs. Handling them is a must

- Exception: Runtime error that disrupts normal flow
- Exception Handling: mechanism to catch and handle errors gracefully
- We need it for program stability, better user experience, debugging and robust applications

◆ 4. Real-World Example

Imagine an ATM software:

- Without exception handling:
 - If DB connection fails → ATM crashes → machine reboots.
- With exception handling:
 - Show "Service temporarily unavailable, please try later" → system keeps running.

Tomorrow I will explore Exception Handling In Depth.